How-To Library

Informatica

How to Configure Big Data Management on Kubernetes

© Copyright Informatica LLC 2019, 2022. Informatica, the Informatica logo, and Big Data Management® are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

You can configure Big Data Management on Kubernetes to optimize resource management and to enable load balancing for the Informatica domain within the containerized environment. This article is written for the Big Data Management administrator responsible for configuring Big Data Management on Kubernetes.

Supported Versions

- Big Data Management 10.2.2 - 10.2.2 HotFix 1

Table of Contents

How Big Data Management Works with Kubernetes. . . . 3
Kubernetes Architecture. . . . 4
Kubernetes Overview. . . . 4
Kubernetes Master Node. . . . 5
Kubernetes Worker Node. . . . 5
Components in the Kubernetes Architecture. . . . 6
Advanced Components in the Kubernetes Architecture. . . . 8
Configuring Big Data Management on Kubernetes Overview. . . . 8
Step 1. Complete the Prerequisites. . . . 9
Verify Port Requirements. . . . 9
Verify System Requirements. . . . 10
Step 2. Create a Kubernetes Cluster on Amazon EKS Cluster. . . . 11
Step 3. Create a Docker Image for Big Data Management 11
Step 4. Create Volumes to Mount on Pods. . . . 12
Step 5. Label Nodes. . . . 12
Step 6. Edit the Properties Files 13
Step 7. Create a Workspace Folder to Store the YAML Files. . . . 13
Step 8. Create ConfigMaps for Pods. . . . 14
Step 9. Create a Network File System. . . . 14
Step 10. Share the License Key for the Nodes on the NFS Mount. . . . 14
Step 11. Create a Secret to Secure the Password and Key Pass Phrase. . . . 15
Step 12. Create a Database Pod. . . . 15
Step 13. Create a Pod that Runs on the Informatica Services. . . . 15
Step 14. Expose the Node Ports to Communicate with Informatica Server from Outside the Cluster. . . . 16
Step 15. Create a Pod that Runs on Informatica Gateway Node. . . . 16
Step 16. Create a Deployment to Manage Worker Nodes. . . . 17
Step 17. Create Horizontal Autoscaling of Pods Based on CPU Usage Metrics. . . . 17
Step 18. Connect to the Administrator Tool. . . . 17
Step 19. Expose Node Ports for Informatica Services with Kubernetes Services. . . . 18
Step 20. Connect to the Domain from the Developer tool. . . . 19

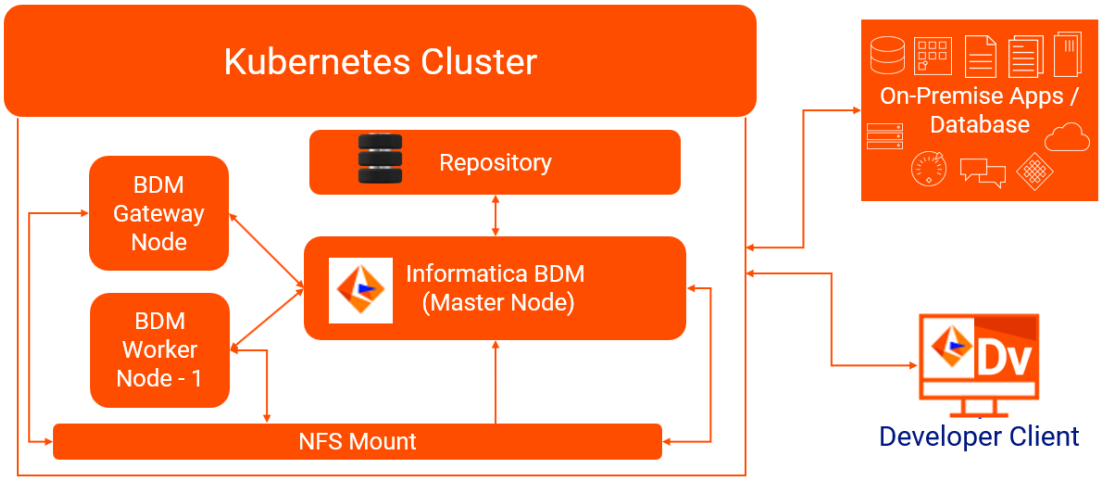
Troubleshooting. 19

How Big Data Management Works with Kubernetes

To create and deploy a self-contained Big Data Management application with a scalable and distributed environment, you can integrate Big Data Management with Kubernetes.

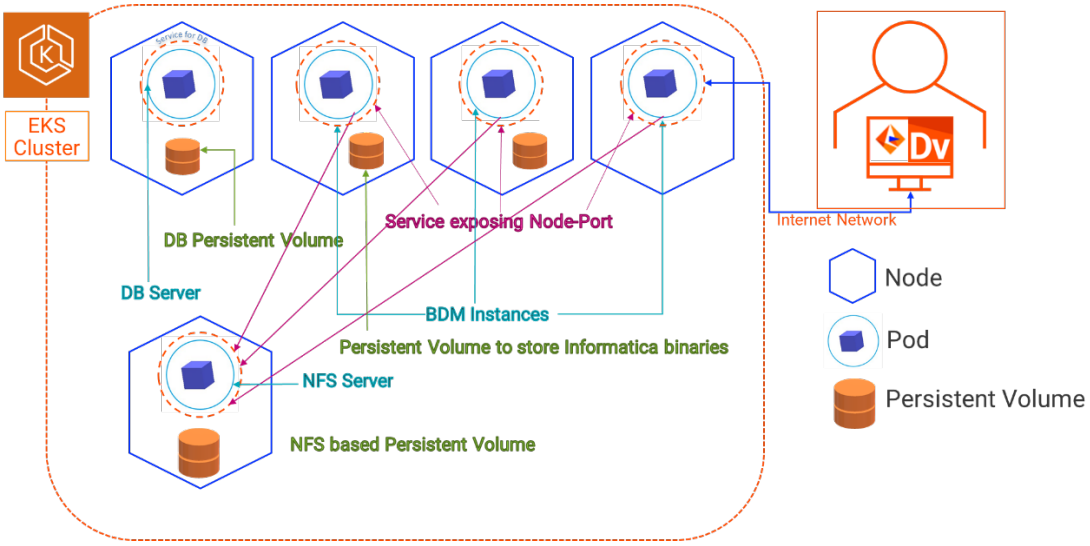
You can host an NFS server to share a mount across all the nodes and to keep the license key file available to the Informatica domain. If a database is not available, create a pod to host the database. Create a pod for Big Data Management master gateway node and deploy the gateway nodes once the master gateway node is up and running. You can connect through the Developer tool as required when the configuration setup is complete.

The following image displays the Big Data Management on Kubernetes cluster:



The diagram shows how a Kubernetes cluster comprises two Big Data Management gateway nodes that share an NFS mount, which is linked to the Informatica Big Data Management master node. The master gateway node is connected to the repository. The Model Repository Service can be configured on any of the nodes. The Kubernetes cluster connects to the on-premises applications or databases and also connect to the client, Informatica Developer.

The following image shows the Big Data Management architecture on Kubernetes that shows the internal communication among pods:



The diagram shows several pods on separate Kubernetes cluster nodes. Each pod resides within a node. One node contains a pod to store the database server and also a database persistent volume. The remaining nodes refer to Big Data Management installation on Kubernetes that exposes the NodePort and helps to interact with the Informatica Developer with the internet network. One node connects to the Big Data Management installation on Kubernetes with the exposed ports and contains an NFS server pod and also an NFS based persistent volume.

Kubernetes Architecture

Kubernetes (K8s) is an open source system to manage containerized applications across multiple hosts, providing basic mechanisms for deployment, maintenance, and scaling of applications.

Kubernetes architecture comprises of kubectl, which is a command line interface that runs commands against the Kubernetes clusters. Kubernetes also has an API server that runs on the master node. ETCD is a distributed key-value store. The Kubernetes state is defined in ETCD. Besides the master node, the Kubernetes architecture contains one or more worker nodes. The workloads run on the worker nodes.

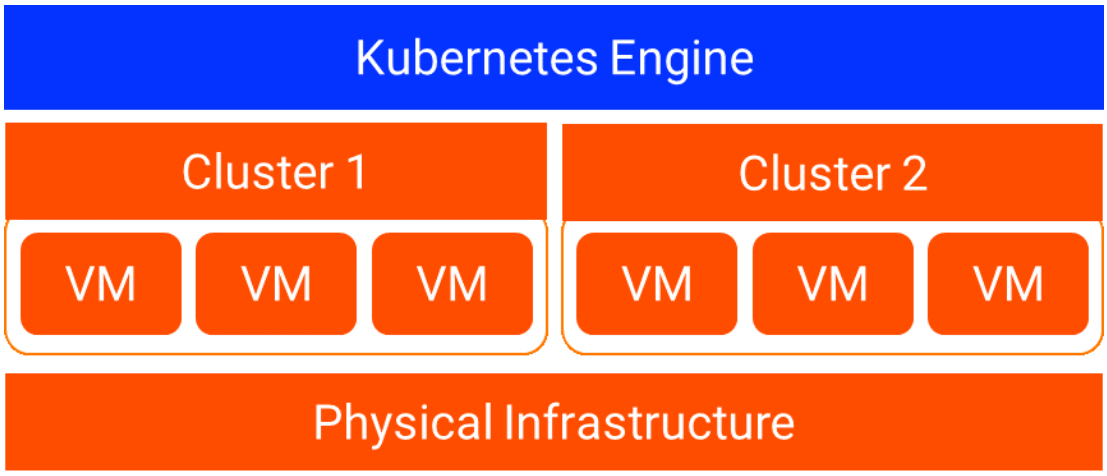
You can specify declarative artifacts for Kubernetes, which are defined using yaml files and these yaml definitions are submitted to the master. Depending on the definition, constraints, and rules, the Kubernetes master node schedules pods or artifacts submitted in one of the nodes. Similar to any distributed computing architecture, the master manages the front end operations and the nodes come together to form a cluster. The registry, either public or private, can centrally stores the docker images, such as a docker hub or a Google container registry.

Kubernetes Overview

A Kubernetes engine is a cluster comprised of one or more master nodes and multiple worker nodes.

A node is a VM hosted on a platform or a server. The node is the component that provides the compute power to the Kubernetes cluster. It is also possible to have a single node master and worker setup, where the master and worker are the same node.

The following diagram illustrates the Kubernetes components, such as the Kubernetes master that connects to the different nodes:



The diagram shows a Kubernetes engine with six VMs and two clusters. Each cluster contains three VMs. Kubernetes resides on the VMs. You can use Kubernetes to determine the VMs to deploy the Big Data Management containers based on storage and compute capability.

Kubernetes Master Node

The Kubernetes cluster master runs the Kubernetes control plane processes, including the Kubernetes API server, scheduler, and core resource controllers.

The following master components are required on a Kubernetes cluster:

kube-apiserver

Master component that exposes the Kubernetes API for all operations.

The master is the unified endpoint for the cluster. All interactions with the cluster are done through the Kubernetes API calls, and the master runs the Kubernetes API server process to handle those requests. The cluster master's API server process is the hub of all communication for the cluster. The internal cluster processes, such as the cluster nodes, system and components, and application controllers act as clients of the API server.

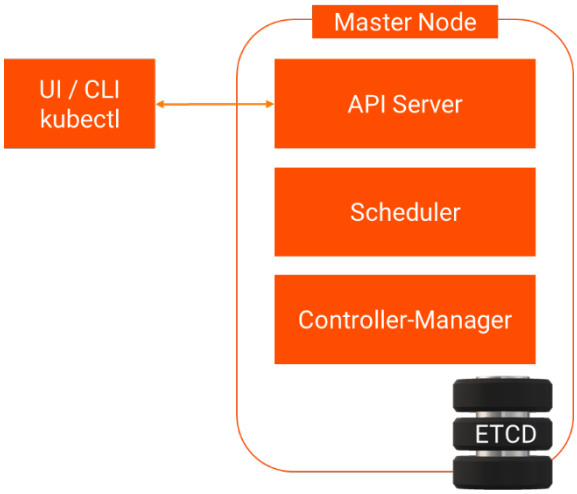
kube-scheduler

Assigns pods to nodes. It also finds free nodes for any workloads before scheduling workloads to the node.

kube-controller-manager

Manages the cluster and interacts with the various APIs in the kube-apiserver.

The following diagram shows the Kubernetes master node components:



The diagram shows how all the services run on the Kubernetes master node. The Kubernetes dashboard or client, kubectl interacts with the API server that runs on the master. The API makes specific calls that enables Kubernetes to process scheduler and controller manager to perform specific tasks. Other than the API calls, you can also communicate to the Kubernetes master through the Kubernetes CLI command known as kubectl.

A scheduler schedules the artifacts such as containers or pods across multiple nodes based on the constraints. A controller is responsible for coordination and manages health of the entire cluster, such that the nodes are up and running and the pods perform correctly in the desired configuration state. ETCD is similar to a database that stores cluster state and configuration data accessed in key value pairs. ETCD can run either on the master or outside the cluster based on the high availability needs.

Kubernetes Worker Node

The worker nodes run the workloads.

The worker node contains kubelet and kube-proxy that both connects to the pods within the docker. Informatica processes along with different OS images that runs on the worker node.

The following node components are needed on a Kubernetes cluster and can also run on master node:

kubelet

A kubelet takes information from the master node and ensures that any pods assigned to it are running and configured in the desired state. All Kubernetes nodes must have a kubelet. The kubelet creates a pod, makes it container ready, and performs a readiness check.

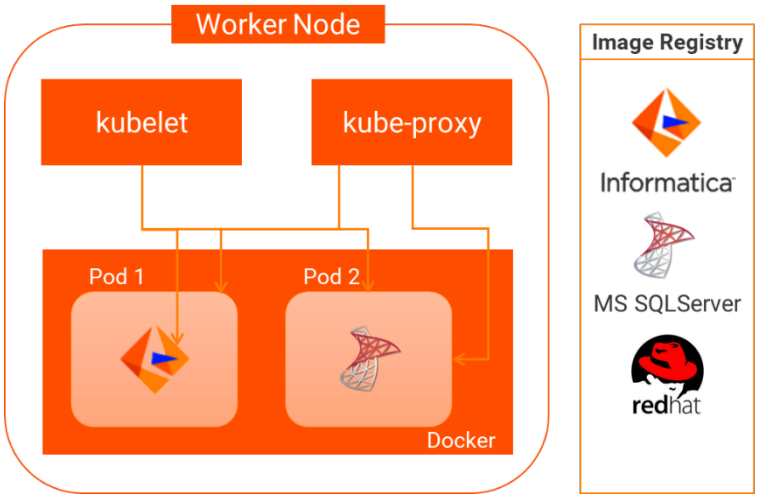
kube-proxy

Watches on all services and maintains the network configuration across all elements of the cluster.

Container runtime

Engine that runs the containers. Containers at runtime such as Docker or RKT are based on the setup configured.

The following image shows the Kubernetes worker node components:



The image shows how the worker node contains the Kubelet and the kube-proxy that is connected to the pods. The kubelet helps to control the state of the files. The docker image registry stores the images of the Informatica binaries, Redhat operating system, and Microsoft SQL Server database. There are two pods that contain different images. Pod 1 contains Informatica domain and services with Red Hat operating system and Pod 2 contains the Microsoft SQL server database. Both the pods resides in a Docker container runtime that you can run as a docker image or a service regular on a system.

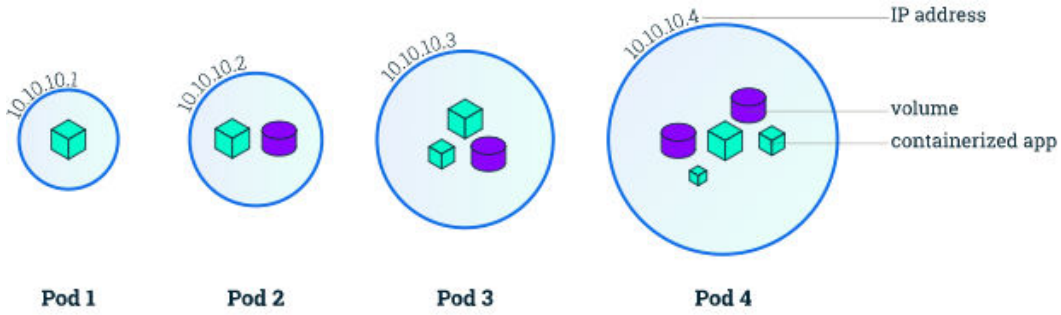
Components in the Kubernetes Architecture

The following components are integral to the Kubernetes architecture:

Pods

A group of one or more containers with shared resources and instructions on how to run containers. Pods can communicate within the cluster using the service IP.

The following image shows the different pods and the various states in which a pod can exist:

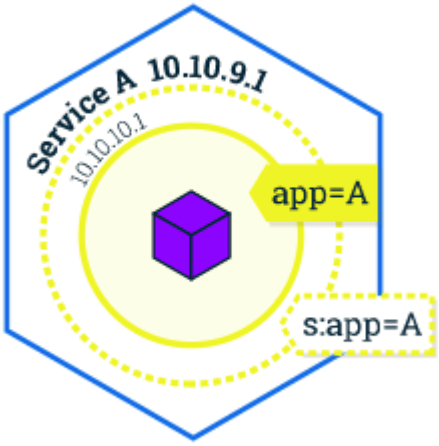


Pod 1 contains a containerized application. Pod 2 contains a containerized application and the volume. Pod 3 contains a containerized application and two volumes. Pod 4 contains two volumes and three containerized applications.

Services

An abstraction that defines a logical set of pods and a policy by which to access the pods. A service in Kubernetes is a rest object that is similar to the pod. Kubernetes services support TCP and UDP for protocols, where the default is TCP. Pods are ephemeral but the services are more stable and you can communicate between different services. It is the Kubernetes service that routes traffic to the specific pod with the same application label.

The following image displays a pod with a service targeting the pod using the application label:



The image shows how a pod with a service A targets the pod with the application label as app=A.

Volumes

A directory that contains data that is accessible to the containers in the pod. A volume lasts longer any containers that run within the pod and data is preserved across container restarts. It also helps maintain the state within the cluster.

Namespaces

Separates workloads from each other along with the resource constraints on each. For example, the pod or service can have multiple developers with a namespace for each developer. To provide a similar workload for each developer, you can configure each namespace state to consume no more than 2 GB RAM.

Advanced Components in the Kubernetes Architecture

The following advanced components pertain to the Kubernetes architecture:

Replication Controller

Create similar pods in a cluster. Replica controllers allow you to create more pods in a cluster when you launch a pod based on availability.

Deployment

Most common form of deployment combines replica set and also helps to roll out or roll back images with a continuous deployment scenario.

StatefulSet

Allows you to easily run databases or anything that requires some persistent state.

DaemonSet

Ensure that there is at least one pod running on each node. You can use daemon sets for logging drivers, monitoring agents, or security agents that you want to run across your cluster.

Job

One or more tasks that you want to run. For example, you have some batch processing jobs to be done and you want to run it once on that batch. After the jobs complete, the pods get destroyed.

Configuring Big Data Management on Kubernetes Overview

You can configure Big Data Management on Kubernetes to optimize resource management and to enable load balancing for the Informatica domain within the containerized environment.

In a multi-node cluster environment, it can be difficult to manage containers that can exist on any of the nodes in a cluster. It also becomes tedious to control and manage the creation of newer containers.

K8s controls the entire container management and orchestration. With K8s, you can package an application in a container image. With K8s, the containers help to decouple the application from the infrastructure. You deploy containers based on operating-system-level virtualization instead of hardware virtualization. With the application-centric management, Kubernetes raises the level of abstraction from running an operating system on virtual hardware to run an application on an operating system using logical resources.

The containers are isolated from each other and from the host. Each container has a separate file system and you can bind the computational resource usage. Containers are easier to build than VMs because the containers are decoupled from the underlying infrastructure and from the host filesystem. The containers are portable across clouds and operating system distributions, such as Ubuntu, RHEL, CoreOS, on-prem, and Amazon EKS.

When you run Big Data Management in a containerized environment and configure the Kubernetes setup, you can optimize the resource management and provide horizontal scaling of pods based on the load provided on the domain.

To run Big Data Management on Kubernetes with Amazon EKS, complete the following steps:

1. Complete the prerequisites.
2. Create a Kubernetes cluster on Amazon EKS.
3. Create a docker image for Big Data Management.
4. Create volumes to mount on pods.
5. Label nodes.
6. Edit the properties files.
7. Create a workspace folder to store the yaml files
8. Create ConfigMaps for pods.

9. Create a network file system.
10. Share the license key for the nodes on the NFS mount.
11. Create a secret to secure the password and key pass phrase.
12. Create a database pod.
13. Create a pod that runs on the Informatica services.
14. Expose the node ports to communicate with Informatica server from outside the cluster.
15. Create a pod that runs on Informatica gateway node.
16. Create a deployment to manage worker nodes.
17. Create horizontal autoscaling of pods based on CPU usage metrics.
18. Connect to the Administrator tool.
19. Expose node ports for Informatica services with Kubernetes services.
20. Connect to the domain from the Developer tool.

Step 1. Complete the Prerequisites

Before you install Kubernetes on Amazon EKS, verify the pre-installation requirements.

Ensure that you meet the following installation prerequisites:

1. Install the unzip utility to create image with Informatica utility.
2. Ensure that the Amazon EKS cluster is running.
3. You must also verify the port and system requirements. For more information, refer the AWS documentation on EKS.
4. Enable RHEL subscription on the host machine used to create Informatica docker image with the RHEL base operating system.
   The host operating system requires RHEL subscription to create the docker image with RHEL as the base operating system and not for running the containers.
5. Download and extract the files listed in the article from the following Informatica Knowledge Base link: https://kb.informatica.com/h2l/HowTo%20Library/1/1268-BDMonKubernetes.zip

You need to be familiar with the eksctl commands. For details on how the eksctl interacts with the cluster see the following AWS Amazon documentation:

https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html

You also need to be familiar with the kubectl commands. For details on how kubectl interacts with the cluster, see the following Kerbernetes documentation:

https://kubernetes.io/docs/reference/kubectl/overview/

Verify Port Requirements

The installer sets up the ports for components in the Informatica domain, and it designates a range of dynamic ports to use for some application services.

You can specify the port numbers to use for the components and a range of dynamic port numbers to use for the application services. Or, you can use the default port numbers provided by the installer. Verify that the port numbers are available on the machines where you install the Informatica domain services.

The following table describes the port requirements:

Port	Description
Database port number	Port number for Microsoft SQL Server. Default is 1433.
Node port number	Port number for the node created during installation. Default is 32005.
Service Manager port	Port number used by the Service Manager on the node. The Service Manager listens for incoming connection requests on this port. Client applications use this port to communicate with the services in the domain. The Informatica command line programs use this port to communicate to the domain. This is also the port for the SQL data service JDBC/ODBC driver. Default is 32006.
Service Manager Shutdown port	Port number that controls server shutdown for the domain Service Manager. The Service Manager listens for shutdown commands on this port. Default is 32007.
Informatica Administrator port	Port number used by Informatica Administrator. Default is 32008.
Informatica Administrator shutdown port	Port number that controls server shutdown for Informatica Administrator. Informatica Administrator listens for shutdown commands on this port. Default is 32009.
Minimum port number	Lowest port number in the range of dynamic port numbers that can be assigned to the application service processes that run on this node. Default is 32014.
Range of dynamic ports for application services	Range of port numbers that can be dynamically assigned to application service processes as they start up. When you start an application service that uses a dynamic port, the Service Manager dynamically assigns the first available port in this range to the service process. The number of ports in the range must be at least twice the number of application service processes that run on the node. Default is 32014 to 32114. The Service Manager dynamically assigns port numbers from this range to the Model Repository Service.
Maximum port number	Highest port number in the range of dynamic port numbers that can be assigned to the application service processes that run on this node. Default is 32114.

Verify System Requirements

Verify that your environment meets the minimum system requirements for the installation process.

The following table lists the system requirements for the installation:

Requirements	Value
Red Hat Enterprise Linux version	7
Disk Space	56 GB. It is the required disk space on VM for image creation.

For more information about product requirements and supported platforms, see the Product Availability Matrix on Informatica Network:

https://network.informatica.com/community/informatica-network/product-availability-matrices

Step 2. Create a Kubernetes Cluster on Amazon EKS Cluster

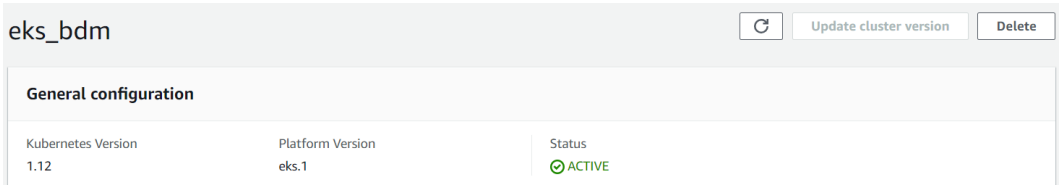
You can use the Amazon EKS Cluster to create a Kubernetes cluster and use Kubernetes as a service with EKS. Use EKS to allow rapid application deployment and to eliminate the need to install, manage, and operate your own Kubernetes clusters.

1. Log in to the following Amazon EKS Cluster console:

   https://console.aws.amazon.com/eks/home#/clusters
2. On the **EKS** page, click **Create Cluster**.
3. On the **Create an EKS cluster** page, enter the cluster details, such as the name of the Amazon EKS cluster and the Kubernetes version to install. Specify the role name for the IAM role to allow Amazon EKS and the Kubernetes control plane to manage the AWS resource.

   You must also specify VPC for your EKS cluster resources and subnets in VPC where the worker nodes run. You must also specify security groups to apply to the EKS managed Elastic Network Interfaces created in the worker node subnets. Set the private access as disabled and the public access as enabled.
4. Click **Create**.

   Cluster creation initiated message appears.
5. Verify that the active cluster has the status as active as seen in the following screenshot:



6. In the **Connect to the cluster** page, you must connect the cluster by authenticating the Amazon SDK and eksctl on the local machine.
7. Enter the command to update kubeconfig in the following syntax in the eksctl command prompt:

   ```
   aws eks --region region update -kubeconfig --name <cluster_name>
   ```
8. Add a worker node to the cluster.

   For more details to add a worker node to the cluster, see the following AWS documentation link:

   https://docs.aws.amazon.com/eks/latest/userguide/getting-started.html#eks-launch-workers

Step 3. Create a Docker Image for Big Data Management

You can create a docker image for Big Data Management with the Informatica Docker utility.

1. The Informatica docker utility installer files are compressed and distributed as a zip file. Use a zip utility to extract the .zip files to a directory on your machine. You can download the .zip file from the Informatica Marketplace.
2. Replace the InfaKubeInstaller.sh in the `<informatica_docker_utility>/source/build/InstallerConfig` path with the InfaKubeInstaller.sh file from the `1268-BDMonKubernetes.zip` file.
3. Replace the Dockerfile_template file in the `<informatica_docker_utility>/source/build/template` path with the Dockerfile_template file from the `1268-BDMonKubernetes.zip` file.

   The template downloads the Microsoft SQL Server client. The template also installs `wget zip unzip dos2unix openssh-server` and `openssh-clients`.
4. Edit the Dockerfile_template and select the terms and agreement to install Microsoft sqlcmd utility against the following text, `ACCEPT_EULA=<Y/N>`, where Y=Yes and N= No

To create an image with the Informatica Docker Utility, see the Informatica How-To Library article "How to Install Informatica Using a Docker Utility": How to Install Informatica Using a Docker Utility

5. Upload the image to the private registry where EKS can access it.

Step 4. Create Volumes to Mount on Pods

Create persistent volumes to mount on pods for the master node, gateway node, and the database.

Before you can create volumes to mount on pods, complete the following prerequisites:

- If you already have a database server running, you can skip the database pod creation and the volume creation.
- To avoid failures, ensure that the pods and the volumes on AWS reside in the same availability zone.

1. Create three persistent volumes to mount on the pods for the Informatica master node, Informatica gateway node, and the database.

   For more information about creating the volumes, see the following AWS documentation link:

   https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-creating-volume.html

2. Verify the location where the nodes are present.

   For more information about using regions and availability zones, see the following AWS documentation link:

   https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html

3. To create the volumes on the regions that contain the nodes, you need to log in to the AWS console and go to **Compute** > **EC2** > **Volumes** > **Create Volume**.
4. Select the volume size based on the requirements.

   For example, create two volumes of 50 GB for the Informatica nodes. For the database, create a volume of 5 GB disk space. If you need to allocate more space, you can allocate additional space when you create the volume.

Step 5. Label Nodes

Create label for nodes based on the volumes created and the node location such that the volume and node location is the same.

Consider ip-172-xx-x-xxx.ec2.internal refers to an EKS worker node that is in us-east-1a and ip-172-xx-yy-yyy.ec2.internal is another EKS worker node that is in us-east-1b.

1. Label the master node in the cluster in the following format:

   ```
   kubectl label nodes <ip-172-xx-x-xxx.ec2>.internal infa=master
   ```

2. Label the gateway node in the cluster in the following format:

   ```
   kubectl label nodes <ip-172-xx-yy-yyy.ec2>.internal infa=gateway
   ```

3. Label the server node in the cluster in the following format:

   ```
   kubectl label nodes <ip-172-xx-x-xxx.ec2>.internal db=server
   ```

Step 6. Edit the Properties Files

Edit the properties files present in the extracted zip file to enter the values for the password, volume ID, master domain, and for the gateway node.

1. To enter the passwords, edit the `Passwords.properties` file.

   You can enter the passwords for the domain, database, Model Repository Service, and the keypass phrase in plain text format.

   Passwords gets encoded and updated in the yaml files required for the Informatica domain configuration.

2. To enter the persistent volumes and regions, edit the `Platform.properties` file.

   You can enter the persistent volume and region for the master and gateway nodes. You can also enter the database and NFS volumes and region.

3. To enter the database and domain configurations, edit the `MasterInput.properties` file.

   You can enter the details to configure the database, such as the database name, port, service name, type, host, and the database name for the Model Repository Service. You can also enter the details to configure the domain, such as the image path, domain name and port, node name, domain user, and the license key location.

4. To enter the gateway node and multiple node configurations, edit the `GatewayInput.properties` file.

   You can enter the domain host name, host IP, and the join domain port for the gateway node. You can also enter the replica value for the multi-node setup.

Step 7. Create a Workspace Folder to Store the YAML Files

1. To create a workspace folder that stores the yaml files, run the `K8s.sh` script.
2. After the `K8s.sh` script completes, verify that the script created the `workspace` folder that contains the following yaml files:

   NFS Server

   Creates the NFS server with the five yaml scripts located in the NFS_Server folder.

   MinToMaxNodePortExpose.yaml

   Exposes the minimum to maximum range of ports for the master node.

   MinToMaxNodePortExposeGateway.yaml

   Exposes the minimum to maximum range of ports for the gateway node.

   database.yaml

   Creates a database server.

   infasecret.yaml

   Creates the secrets required to pass to the Informatica pods.

   multinode_deployment.yaml

   Creates a deployment of the Informatica worker nodes.

   infaserver.yaml

   Creates the Informatica master node.

   joingateway.yaml

   Created the Informatica gateway node.

Step 8. Create ConfigMaps for Pods

You can use ConfigMaps to decouple the configuration artifacts from the image content to keep the containerized applications portable. You can create ConfigMaps and configure pods with the data stored in the ConfigMaps.

You can use the kubectl create configmap to create a Config Map.

In Informatica, the ConfigMap stores the values of the MasterInput.properties and GatewayInput.properties and then provides those values as environment variable values for the Informatica pods.

- To create a ConfigMap from files for the master node with kubectl, enter the following command:

  ```
  kubectl create configmap infa-master-config --from-env-file=MasterInput.properties
  ```

  To get information about the ConfigMap, you can use the kubectl describe and kubectl get commands.

  Note: If you reference a ConfigMap that does not exist, the pod cannot start. Ensure that you create the config map for the gateway node after the Informatica master node is up and running.

Step 9. Create a Network File System

You can create a Network File System (NFS), which is a common container location for the pods to store logs. You can also persist the logs in case the container goes down.

To make license key available to all pods and to effectively run the Data Integration Service on a grid, create the following entities before exposing the Kubernetes service to communicate among the pods:

- Persistent Volume
- Persistent Volume Claim
- NFS server

For details on the entities, see the following link:

https://github.com/kubernetes/examples/tree/master/staging/volumes/nfs

To create the NFS volume, perform the following steps in the `workspace/NFS_Server` folder:

1. Update the `1-nfs-server-pv.yaml` file.
2. Update the `2-nfs-server-rc.yaml` file.
3. Update the `3-nfs-server-service.yaml` file.
4. To get the IP of the NFS Server, enter the following command:

   ```
   kubectl get services -o wide
   ```

5. Add the correct IP in the following file as the value for the server:

   ```
   4-nfs-pv.yaml
   ```

6. Update the `4-nfs-pv.yaml` file.
7. Update the `5-nfs-pvc.yaml` file.

Step 10. Share the License Key for the Nodes on the NFS Mount

1. To ensure that the nfs-server pod runs, run the `kubectl get pods -o` command and verify the run state in the status column.
2. Enter the following kubectl cp command to copy the license key file to the mount location:

   ```
   kubectl cp <file_name> <nfs-server_pod_name>:/exports/<file_name>
   ```

   The shared key becomes available for all the nodes with the NFS mount.

Step 11. Create a Secret to Secure the Password and Key Pass Phrase

You can create a secret to secure the sensitive information, such as OAuth tokens and SSH keys. Instead of storing the information in a pod definition or in a docker image, you can store the secret securely.

Ensure that you create the following secrets before using it with a pod:

- Domain password. Password for the domain administrator.
- Database password. Password for the database user account.
- Encryption key pass phrase. Keyword to create an encryption key to secure the sensitive data in the domain.

1. Enter the following command in the Kubernetes command prompt under the `workspace` directory:

   ```
   kubectl create -f infasecret.yaml
   ```

2. To authenticate your private image registry, create a secret and pass it in the YAML file required for Informatica server pod creation.

   For more details about the secure pod creation, refer the following Kubernetes documentation links:

   https://kubernetes.io/docs/concepts/containers/images/#specifying-imagepullsecrets-on-a-pod

   https://kubernetes.io/docs/concepts/containers/images/#using-a-private-registry

Step 12. Create a Database Pod

You can create a disk for the database creation. If a container goes down, the persisted data would be available on another database.

1. On the Navigation Menu of the AWS EKS console, go to **Compute Engine** > **Disks** > **CREATE DISK**
2. To create the database pod with the Microsoft SQL Server 2017 docker image, enter the required details in the `database.yaml` file under the `workspace` folder.

   The Kubernetes service exposes the required pods for communication.
3. To set up the database server and create a database, complete the steps listed in the `db_creation.txt` file.

Step 13. Create a Pod that Runs on the Informatica Services

Create a pod that runs on the Informatica services and ensure that the pod is available.

Before you create a pod, verify that the database server is running.

1. Create the Informatica master pod.

   To create the Informatica master pod, run the following command under the `workspace` directory:

   ```
   kubectl create -f infaserver.yaml
   ```

2. Ensure that the node pods are available from 32005 to 32012. If the ports are unavailable, manually edit the node ports with a constraint such that both the target ports and the node ports have a 1-to-1 mapping. Otherwise, you cannot connect using the Developer tool.

   For example:

   ```
   - name : p1
   protocol: TCP
   port: 32005
   targetPort: 32005
   nodePort: 32005
   ```

After verifying that the pod is in a running state, monitor the installation process. You can do this by logging in to the pod and by checking the installation log present in the installation location:

`/home/Informatica/Server/`

The installation log has the following convention:

`Informatica_<version>_Services_<TIME_STAMP>.log`

After the installation completes successfully, the installation status shows as "SUCCESS".

Step 14. Expose the Node Ports to Communicate with Informatica Server from Outside the Cluster

The Kubernetes service exposes certain ports. By default, the pod uses the node ports from 32005 to 32012, so ensure these ports are available. If unavailable, the node ports have to be edited manually after ensuring that the target port and node port has a one to one mapping. Otherwise, you cannot connect using the Developer tool.

You can run the following command in the workspace directory:

```
kubectl create -f MinToMaxNodePortExpose.yaml
```

Step 15. Create a Pod that Runs on Informatica Gateway Node

Ensure that the Informatica domain is running.

1. Edit the `GatewayInput.properties` file and add the pod IP of the infamaster pod.
2. To locate the IP address of the infamaster pod, enter the following command:

   ```
   kubectl get pods -o wide
   ```

   The IP address gets assigned when the container is created.

   You can see the IP in the following sample output:

   ```
   NAME                              READY     STATUS     RESTARTS     AGE
   IP
   NODE
               NOMINATED NODE
   infamaster        1/1               Running 0
   77m     172.xx.xx.xxx        ip-172-xx-x-xxx.ec2.internal    <none>
   ```

3. Copy the IP and update the DOMAIN_HOST_IP field in the GatewayInput.properties file.
4. To create a configuration map for the gateway node, enter the following command:

   ```
   kubectl create configmap infa-gateway-config --from-env-file=GatewayInput.properties
   ```

5. To create an Informatica gateway node from the workspace directory, enter the following command:

   `kubectl create -f joingateway.yaml` file.

6. After verifying that the pod is in a running state, monitor the installation process.

   You can monitor the installation by logging into the pod and by checking the installation log present in the installation location:

   ```
   /home/Informatica/Server/
   ```

   The installation log has the following convention:

   ```
   <Informatica_<version>_Services_<TIMESTAMP>.log
   ```

   After the installation completes successfully, the installation status shows as "SUCCESS."

   After joining the node, you can log into the Administrator tool to verify the same.

   To know more information about the services created, you can monitor the infa_setup.log. You can run an interactive bash shell on the running pod with the kubectl exec command and then check the infa_setup.log under the / directory.

   You can enter the kubectl exec command in the following format:

   ```
   kubectl exec -it <pod_name> -- /bin/bash
   ```

Step 16. Create a Deployment to Manage Worker Nodes

Create a deployment for two Informatica worker nodes in the grid with the Data Integration Service process enabled.

- Run the following command to create a deployment in the workspace directory:

  ```
  kubectl create -f multinode_deployment.yaml
  ```

  This creates a group of two Informatica worker nodes added to the grid with the Data Integration Service process enabled.

Step 17. Create Horizontal Autoscaling of Pods Based on CPU Usage Metrics

With the Horizontal Pod Autoscaler, you can automatically scale the number of pods in a deployment or replica set based on the noted CPU utilization.

- Enter the following command:

  ```
  kubectl autoscale deployment multimode-deployment --min=2 --max=5 --cpu-percent=80
  ```

  This creates horizontally scalable Informatica worker nodes that are added to the grid with the Data Integration Service process enabled.

Step 18. Connect to the Administrator Tool

To connect to the Administrator tool, you need the node IP on which the infaserver runs and the nodePort of the pod.

1. Enter the following command to list the status of all pods:

   ```
   kubectl get pod infaserver -o wide
   ```

   The output displays the node name on which the infaserver runs.

2. To get the node's IP address, enter the following command:

   ```
   kubectl get nodes -o wide
   ```

   The output lists all the available nodes and the corresponding external IP. Note the IP address of the node that has the new-infaserver pod running.

3. On the Windows machine, add the following entry to the hosts file in the `C:\Windows\System32\drivers\etc\hosts` path, and save the file:

   ```
   <Node_External_IP> <tab or space> <pod_name>
   ```

   For example:

   ```
   37.100.167.23 infaserver
   ```

4. To access the Administrator tool in a browser, enter the pod name followed by the NodePort number.

   Use the following format: http://<pod-name>:<NodePort>

   For example, http://new-infaserver:32005

   Default Administrator tool node port number is 32008.

Step 19. Expose Node Ports for Informatica Services with Kubernetes Services

By default, Informatica services sets the minimum and maximum of ports that the Informatica services uses to assign port numbers to any new services created. For example, if the minimum and maximum port range is 30014 to 30114, then the Informatica services assign ports within the mentioned range.

1. You can expose ports with the default or with the custom port name and service name.

   - For the default port range, you can run the MinToMaxNodePortExpose.yaml file for the Informatica master node. For the default port range, you can run the MinToMaxNodePortExposeGateway.yaml file for the Informatica gateway node. To run the yaml files, you can run the command: `kubectl create -f MinToMaxNodePortExpose.yaml`
   - For the custom port range, you can use the `NodePortRangeExpose-<ServiceName>.yaml` file for the required service.

2. For custom port range, run the Java program, `ExposePort.jar` so that the jar file creates the yaml files required to expose the ports for the services required on the gateway and master nodes.

   The default gateway node ports are from 30005 - 30012 and 30014 - 30114. The default master node ports are from 32005 - 32012 and 32014 - 32114.

   For example, enter the following command to expose the gateway node ports for the services required:

   ```
   java -jar ExposePort.jar 30014 30114 node-min-max-service-gateway infa-gateway
   ```

   If the specified NodePorts are unavailable, manually enter the NodePorts where the targetPort can correspond with the NodePort.

   For example, enter the following command to expose the master node ports for the services required:

   ```
   java -jar ExposePort.jar 32014 32114 node-min-max-service infa-master
   ```

   This creates the `NodePortRangeExpose-<ServiceName>.yaml` file with the available ranges to expose.

3. To expose the custom ports by creating the service, enter the following command under the `workspace` directory:

   ```
   kubectl create -f NodePortRangeExpose-<ServiceName>.yaml
   ```

   The service targets all pods with the label set in the purpose property in the `NodePortRangeExpose-<ServiceName>.yaml` file.

Step 20. Connect to the Domain from the Developer tool

Perform the following steps to set the entry in the hosts file:

1. Add the node and pod entries correctly to the `C:\Windows\System32\drivers\etc\hosts` file on the machine where the Developer tool is installed.
2. Enter the following command to list the status of all the pods:

   `kubeclt get pod new -infaserver -o wide.`
   The output displays the node name on which it is running.
3. To get the node's IP address, enter the following command:

   `kubectl get nodes -o wide`
   The output lists all the available nodes and the external IP address.
4. Get the IP address of the node that has the new infaserver pod running.
5. On a Windows machine, add the following entry to the `C:\Windows\System32\drivers\etc\hosts` file:
   `<Node_External_IP> <tab or space> <pod_name>.`
   For example, `37.100.167.23 new-infaserver.`
6. Save the file after adding all the entries.

To connect to the domain from the Developer tool, perform the following steps:

1. In the **Object Explorer** view, right-click the Model Repository Service associated with the Model repository to which you want to connect.
2. Select **Connect**.

   The **Connect to Domain** dialog box appears.
3. Enter the user name and password.
4. Click **OK**.

   The Developer tool connects to the Model repository.

Troubleshooting

Gateway node fails to join to the master node

This issue occurs when the installer runs the ping gateway command before the entry of the gateway pod appears in the `/etc/hosts` file of the master node.

You can start the node after the host entry of the gateway pod is added to the /etc/hosts file of the pod that runs the master node.

To get the IP address of the gateway pod, run the following command:

```
kubectl get pods -o wide
```

Locate the IP address of the new gateway pod. The IP gets assigned after the container has been created.

Authors

Sujitha Alexander

Debjyoti Thakur

Ankur Vijayvargiya